TUTORIAL ASSESSMENT · SEMESTER 2 · 2026

Programming Principles B

Tutorial Question Paper 3

PPB115D / PPG115D / PPBF15D / PPGF15D · DPIF20 / DPMC20 / DPRS20 / DPIT20

YEAR	SEMESTER	ASSESSMENT	PAPER
2026	**2**	**Tutorial paper 3**	**Computer based**
DURATION	FULL MARKS	SECTION A	SECTION B
3 hours	**120**	**35 marks**	**85 marks**

STUDENT DETAILS

SURNAME AND INITIALS _____

STUDENT NUMBER _____

TUTORIAL GROUP _____

TUTOR _____

SPECIAL REQUIREMENTS

- [x] Computer with the Java JDK installed
- [x] Notepad++ or another text editor
- [ ] Non-programmable pocket calculator
- [ ] Internet access
- [ ] Annexures (none)

MARK ALLOCATION

Q	Topic	Marks	Obtained
1	True / False	10	
2	Multiple choice	10	
3	Short questions	15	
4	Concrete class	20	
5	Arrays and static methods	65	
	Total	**120**	

INSTRUCTIONS TO CANDIDATES

1. Answer **all** questions. Check each question for additional instructions.
2. Create a folder on the **D:** drive named with your student number, and save all your work in it.
3. Type your Section A answers in a Word or Notepad document called `SectionA_StudentNumber`.
4. Write Questions 4 and 5 in Notepad++. Compile and run each program before you hand it in.
5. Do not change the class names, method names, parameter lists or return types given in the UML diagrams.

Total number of pages including the cover page: **13**

Total number of annexures: **0**

SECTION A

Theory · 35 marks

QUESTION 1 True / False — 10 marks

Instructions. Write **True** or **False** next to each question number in your Section A document. Each statement is worth 1 mark.

No.	Statement	T / F
1.1	A `do...while` loop tests its condition after the body has executed.	
1.2	The expression `5 / 2` in Java evaluates to 2.5.	
1.3	A variable declared inside the body of a `for` loop can still be used after the loop has ended.	
1.4	A `boolean` method can return the result of a comparison directly, for example `return a > b;`.	
1.5	In Java, a `char` value is written between double quotes.	
1.6	The statement `int[] a = {1, 2, 3};` creates an array with a length of 3.	
1.7	`"Cape Town".length()` returns 9.	
1.8	A method may contain more than one `return` statement.	
1.9	Accessor methods are normally declared with the return type `void`.	
1.10	`"abc".equals("ABC")` returns `true`.	

QUESTION 2 Multiple Choice — 10 marks

Instructions. Write only the letter of the correct answer next to each question number. Each question is worth 1 mark.

2.1 What does the following code display?

```java
int n = 1;
while (n < 20)
{
   n *= 3;
}
System.out.println(n);
```

a. 9 **b.** 27
c. 20 **d.** 81

2.2 What is the value of `17 % 5`?

a. 3 **b.** 3.4
c. 2 **d.** 12

2.3 `input` is a `Scanner` object. Which statement reads a whole line of text into `name`?

a. `String name = input.readLine();` **b.** `String name = nextLine(input);`
c. `String name = input.next();` **d.** `String name = input.nextLine();`

2.4 What does the following code display?

```java
int[] v = {2, 4, 6, 8};
int s = 0;
for (int i = 1; i < v.length; i += 2)
{
   s += v[i];
}
System.out.println(s);
```

a. 12 **b.** 8
c. 20 **d.** 6

2.5 What does `"Soweto".toUpperCase().charAt(2)` return?

a. `'w'` **b.** `'W'`
c. `'O'` **d.** `'E'`

2.6 Which is the correct header for a method that receives two `int` values and returns the larger one?

a. `public static void max(int a, int b)`

b. `public static int max(int a, b)`

c. `public static max(int a, int b)`

d. `public static int max(int a, int b)`

2.7 An `int` variable is passed to a method, and the method changes its parameter. What happens to the variable in the calling method?

a. It stays unchanged

b. It changes too

c. It becomes 0

d. The code does not compile

2.8 Which code correctly swaps `a[i]` and `a[j]`?

a. `a[i] = a[j]; a[j] = a[i];`

b. `temp = a[i]; a[j] = temp; a[i] = a[j];`

c. `temp = a[i]; a[i] = a[j]; a[j] = temp;`

d. `a[i] == a[j];`

2.9 Which statement creates an object of a class called `Gym` with its default constructor?

a. `Gym g = Gym();`

b. `Gym g = new Gym();`

c. `new g = Gym();`

d. `Gym g = new Gym;`

2.10 How should code outside a class read a `private` instance variable of that class?

a. Through a public accessor method

b. By using its name directly

c. By declaring it again

d. By using the `new` keyword

QUESTION 3 Short Questions — 15 marks

Instructions. Write only the required Java statement or statements. You do not need to write a full program. **Suggested time:** 20 minutes.

3.1 Write only the required statement(s) for each scenario.

3.1.1 An `int` variable called `total` holds 0. Write a `for` loop that adds the numbers 1 to 100 to `total`. **(2)**

3.1.2 Write a `for` loop that displays every element of an `int` array called `ages`, each on its own line. **(2)**

3.1.3 Write an `if` statement that displays "Pass" if an `int` variable called `mark` is 50 or more, and "Fail" otherwise. **(2)**

3.2 Consider the following declaration.

```java
String text = "Many hands make light work";
```

3.2.1 Use the `indexOf()` and `substring()` methods to extract the word `hands` and store it in a `String` called `word`. **(2)**

3.2.2 Write a statement that calls a method called `isValidText`, passes `text` to it, and stores the `boolean` value it returns in a variable called `valid`. **(2)**

3.2.3 Write the statements that count the spaces in `text`, and then display the number of words (spaces + 1). **(5)**

END OF SECTION A

SECTION B

Practical programming · 85 marks

QUESTION 4 Concrete Class — 20 marks

Preparations. Create the folder `D:\StudentNumber\Question4\` and save two files in it: `GymMembershipApp.java` and `TestGymMembershipApp.java`. Complete the partially written code in Tables 4.5 and 4.6.

Short program: Gym Membership Application

FitLife is a gym in Durban. It needs a program that takes the number of months a member signs up for and their membership plan, works out the total fee and the average cost per month, and then displays a membership summary.

TABLE 4.1 MEMBERSHIP PLANS

Plan	Fee per month
Basic	R250.00
Premium	R420.00
Student	R180.00

RULES

Total fee = fee per month × months
12 months or more: 10% discount on the total
Monthly average = total fee ÷ months
Unknown plan → total fee is **R0.00**

TABLE 4.2 UML DIAGRAM OF GYMMEMBERSHIPAPP

GymMembershipApp
- months : int - planType : String - monthlyAverage : double
+ GymMembershipApp() + setMonths(int) : void + setPlanType(String) : void + determineTotalFee() : double + calculateMonthlyAverage(double) : void + toString(double) : String

TABLE 4.3 DESCRIPTION OF THE GYMMEMBERSHIPAPP CLASS

Member / method	Description	Marks
`months`, `planType`, `monthlyAverage`	Private instance fields.	2
`GymMembershipApp()`	Default constructor. Assigns default values to all three instance fields.	2
`setMonths()`, `setPlanType()`	Mutator methods.	2

Member / method	Description	Marks
`determineTotalFee()`	Determines and returns the total fee, using Table 4.1 and the discount rule. The plan must not be case sensitive.	5
`calculateMonthlyAverage(double)`	Receives the total fee. Calculates the average cost per month and stores it in `monthlyAverage`.	2
`toString(double)`	Receives the total fee. Returns the state of the object as one `String`, formatted as in Figures 4.1 to 4.3.	2

TABLE 4.4 DESCRIPTION OF THE TESTGYMMEMBERSHIPAPP CLASS

Method	Description	Marks
`main()`	1. Create an object called `member` of the `GymMembershipApp` class, using the default constructor. 2. Let the user enter the number of months and the plan. 3. Set the months and the plan by calling the mutator methods. 4. Determine the total fee, then calculate the monthly average. 5. Display the membership summary by calling `toString()`.	5

TABLE 4.5 PARTIALLY COMPLETED CODE

GYMMEMBERSHIPAPP.JAVA

```
public class GymMembershipApp
{
   // 4.1 Declare the instance fields
   // 4.2 Default constructor
   // 4.3 Mutator methods
   // 4.4 determineTotalFee()
   public double determineTotalFee()
   {
      final double DISCOUNT_RATE = 0.10;
      double total = 0.0;

      // complete the method

      return total;
   }
   // 4.5 calculateMonthlyAverage(double)
   // 4.6 toString(double)
}
```

TABLE 4.6 PARTIALLY COMPLETED CODE

TESTGYMMEMBERSHIPAPP.JAVA

```
import java.util.Scanner;

public class TestGymMembershipApp
{
   public static void main(String[] args)
   {
      Scanner input = new Scanner(System.in);

      // 4.7 Complete the main method
   }
}
```

Sample output

FIGURE 4.1 PREMIUM PLAN, NO DISCOUNT

Console output

```
Enter the number of months: 6
Enter the plan (Basic/Premium/Student): Premium

*** FitLife Membership Summary ***
Plan            : Premium
Months          : 6
Monthly average : R420.00
Total fee       : R2520.00
```

FIGURE 4.2 BASIC PLAN, WITH DISCOUNT (PLAN TYPED IN LOWER CASE)

Console output

```
Enter the number of months: 12
Enter the plan (Basic/Premium/Student): basic

*** FitLife Membership Summary ***
Plan            : basic
Months          : 12
Monthly average : R225.00
Total fee       : R2700.00
```

FIGURE 4.3 STUDENT PLAN, WITH DISCOUNT

Console output

```
Enter the number of months: 15
Enter the plan (Basic/Premium/Student): Student

*** FitLife Membership Summary ***
Plan            : Student
Months          : 15
Monthly average : R162.00
Total fee       : R2430.00
```

QUESTION 5 Arrays and Static Methods — 65 marks

Preparations. Create the folder `D:\StudentNumber\Question5\` and create a new file in it called `StokvelManagementApp.java`.

Stokvel Management Application

Masakhane Stokvel is a savings club in Galeshewe. Members pay a monthly contribution and take turns to receive a payout. The records are kept in a notebook, so payout decisions are slow and members often dispute them.

The club wants a program that decides each member's payout status from their years in the group and the number of payments they have missed. Table 5.1 gives the rules and Table 5.2 gives sample data for six members.

TABLE 5.1 CRITERIA

Years in the group	Missed payments	Payout status
5 or more	0 to 1	**Priority Payout**
5 or more	2 to 3	**Normal Payout**
5 or more	4 or more	**Under Review**
Fewer than 5	0	**Normal Payout**
Fewer than 5	1 to 3	**Waiting List**
Fewer than 5	4 or more	**Under Review**

TABLE 5.2 SAMPLE DATA

Member ID	Name	Monthly contribution (R)	Years in group	Missed payments	Payout status
M201	Thandi Mokoena	500.00	0	0	
M202	Sipho Dlamini	750.00	0	0	
M203	Lerato Nkosi	300.00	0	0	
M204	Pieter van Wyk	1000.00	0	0	
M205	Zanele Khumalo	650.00	0	0	
M206	Themba Sithole	400.00	0	0	

The program must let staff:

- display the details of all members;

- search for one member's payout status by member ID;
- sort the members by monthly contribution, from highest to lowest;
- see which members pay the lowest and the highest contribution;
- display the average monthly contribution.

TABLE 5.3 UML DIAGRAM OF STOKVELMANAGEMENTAPP (ALL METHODS ARE STATIC)

StokvelManagementApp

`+ main(String[]) : void`
`+ populateYearsInGroup(int[]) : void`
`+ populateMissedPayments(String[], int[]) : void`
`- validateMissedPayments(int) : boolean`
`+ getUserOption() : char`
`+ determinePayoutStatus(int[], int[], String[]) : void`
`+ searchMember(String, String[]) : int`
`+ displayAllMembers(String[], String[], double[], int[], int[], String[]) : void`
`+ sortByContribution(String[], String[], double[], int[], int[], String[]) : void`
`+ determineLowestContribution(double[]) : int`
`+ determineHighestContribution(double[]) : int`
`+ calculateAverageContribution(double[]) : double`

TABLE 5.4 DESCRIPTION OF THE METHODS

No.	Method	Description	Marks
5.4.1	`populateYearsInGroup()`	Receives the years-in-group array. Fills it with random whole numbers from 1 to 10, using a method of the `Random` class.	3
5.4.2	`populateMissedPayments()`	Receives the member ID array and the missed payments array. For each member: • prompt for the missed payments (Figure 5.1); • call `validateMissedPayments()` to check it; • if it is invalid, display "*Invalid number. Please enter a value from 0 to 12.*" and let the user enter it again until it is valid; • store the valid value in the array.	7
5.4.3	`validateMissedPayments()`	Receives one value. Returns `true` if it is from 0 to 12 (inclusive), otherwise `false`.	3
5.4.4	`getUserOption()`	Displays the menu in Figure 5.2, lets the user enter an option and returns it.	3
5.4.5	`determinePayoutStatus()`	Receives three arrays: years in the group, missed payments and payout statuses. Uses Table 5.1 to determine each member's status and stores it in the payout status array.	7

No.	Method	Description	Marks
5.4.6	`searchMember()`	Receives the member ID to look for and the member ID array. Returns the index where it was found, or -1 if it was not found. The search must not be case sensitive.	4
5.4.7	`displayAllMembers()`	Receives all six arrays. Displays all members in columns, as in Figure 5.3.	3
5.4.8	`sortByContribution()`	Receives all six arrays. Uses the **Bubble Sort** algorithm to sort the members by monthly contribution in **descending** order (highest first). The arrays are parallel, so whenever two contributions are swapped, the same two positions must be swapped in the other five arrays.	8
5.4.9	`determineLowestContribution()`	Receives the contribution array. Returns the **index** of the lowest monthly contribution. If there is a tie, it returns the first one.	2
5.4.10	`determineHighestContribution()`	Receives the contribution array. Returns the **index** of the highest monthly contribution. If there is a tie, it returns the first one.	2
5.4.11	`calculateAverageContribution()`	Receives the monthly contribution array. Calculates and returns the average monthly contribution as a `double`, keeping the decimals.	3

5.5 Create the main method (20)

1. Declare six parallel arrays: `memberIDs`, `memberNames`, `contributions`, `yearsInGroup`, `missedPayments`, `payoutStatuses`.
2. Initialise `memberIDs`, `memberNames` and `contributions` with the data in Table 5.2.
3. Fill `yearsInGroup` by calling `populateYearsInGroup()`.
4. Fill `missedPayments` by calling `populateMissedPayments()`.
5. Fill `payoutStatuses` by calling `determinePayoutStatus()`.
6. Call `getUserOption()` and take the action in Table 5.5 for the option entered. **Use a `switch` statement.**
7. Repeat step 6 until 6 or X is entered.
8. When 6 or X is entered, display "Goodbye" at the end of the program.

TABLE 5.5 MENU ACTIONS (STEPS 1 TO 5 CARRY THE REMAINING 4 MARKS)

Option	Action	Marks
1 / D	Display all members by calling `displayAllMembers()` (Figure 5.3).	2
2 / S	Prompt for a member ID and call `searchMember()`. If it is found, display the member's details (Figure 5.4). Otherwise display a suitable message (Figure 5.5).	5

Option	Action	Marks
3 / O	Call `sortByContribution()`, then display the sorted members by calling `displayAllMembers()` (Figure 5.6).	2
4 / L	Call `determineLowestContribution()` and `determineHighestContribution()`. Use the returned indexes to display each member's name and contribution (Figure 5.7).	3
5 / A	Call `calculateAverageContribution()` and display the result to 2 decimal places (Figure 5.8).	2
6 / X	Exit the program (Figure 5.9).	1
Other	Display an error message, then display the menu again (Figure 5.10).	1

Sample output

Years in the group are random, so your values will differ. These figures use years 7, 3, 5, 2, 9, 4.

FIGURE 5.1 ENTERING MISSED PAYMENTS (WITH TWO INVALID ENTRIES)

Console output

```
Enter the number of missed payments for M201: 0
Enter the number of missed payments for M202: 2
Enter the number of missed payments for M203: -1
Invalid number. Please enter a value from 0 to 12.
Enter the number of missed payments for M203: 3
Enter the number of missed payments for M204: 0
Enter the number of missed payments for M205: 15
Invalid number. Please enter a value from 0 to 12.
Enter the number of missed payments for M205: 5
Enter the number of missed payments for M206: 1
```

FIGURE 5.2 MENU

Console output

```
===== MASAKHANE STOKVEL =====
1/D - Display all members
2/S - Search for a member
3/O - Sort by contribution
4/L - Display lowest and highest contribution
5/A - Display average monthly contribution
6/X - Exit
Enter your option:
```

FIGURE 5.3 OPTION 1/D

Console output

```
ID     Member            Contribution Years  Missed  Status
M201   Thandi Mokoena    500.00       7      0       Priority Payout
M202   Sipho Dlamini     750.00       3      2       Waiting List
M203   Lerato Nkosi      300.00       5      3       Normal Payout
M204   Pieter van Wyk    1000.00      2      0       Normal Payout
M205   Zanele Khumalo    650.00       9      5       Under Review
M206   Themba Sithole    400.00       4      1       Waiting List
```

FIGURE 5.4 OPTION 2/S, FOUND

Console output

```
Enter the member ID to search for: m204
ID           : M204
Name         : Pieter van Wyk
Contribution : R1000.00
Missed       : 0
Status       : Normal Payout
```

FIGURE 5.5 OPTION 2/S, NOT FOUND

Console output

```
Enter the member ID to search for: M999
Member ID M999 was not found.
```

FIGURE 5.6 OPTION 3/O

Console output

```
Members sorted by contribution (highest to lowest):
ID     Member             Contribution Years  Missed  Status
M204   Pieter van Wyk     1000.00      2      0       Normal Payout
M202   Sipho Dlamini      750.00       3      2       Waiting List
M205   Zanele Khumalo     650.00       9      5       Under Review
M201   Thandi Mokoena     500.00       7      0       Priority Payout
M206   Themba Sithole     400.00       4      1       Waiting List
M203   Lerato Nkosi       300.00       5      3       Normal Payout
```

FIGURE 5.7 OPTION 4/L

Console output

```
Lowest contribution  : Lerato Nkosi (R300.00)
Highest contribution : Pieter van Wyk (R1000.00)
```

FIGURE 5.8 OPTION 5/A

Console output

```
Average contribution: R600.00
```

FIGURE 5.9 OPTION 6/X

Console output

```
Enter your option: X
Goodbye
```

FIGURE 5.10 ANY OTHER KEY

Console output

```
Enter your option: 7
Invalid option. Please choose 1/D, 2/S, 3/O, 4/L,
5/A or 6/X.
```

END OF PAPER

Section A: **35** Section B: **85** Full marks: **120**